

ToyGPU-4

A first-principles, CUDA-compatible toy GPU with 4 SMs
Microarchitecture, SystemVerilog RTL guide and verification plan (front-end only)

Scope: architecture + front-end (RTL design, verification, implementation plan). FPGA prototyping is intentionally out of scope for this version. All numbers are toy choices, exposed as SystemVerilog parameters, so they can be changed later.

How to read this: Sections 1 to 3 build intuition (what a GPU is, the toy spec, the life of a kernel). Sections 4 to 9 are the microarchitecture. Sections 10 to 11 cover the ISA and the CUDA software path. Sections 12 to 14 are the SystemVerilog HDL guide, the verification plan and the roadmap. Every diagram is hand-drawn style and also available as a PNG in the diagrams folder.

1. GPU from first principles

A CPU core is built to finish one thread fast: deep caches, branch prediction, out-of-order execution. A GPU is built to finish millions of threads with high total throughput. It gets there with three ideas:

1. Many simple lanes instead of a few complex cores. Silicon goes to ALUs and registers, not to speculation.
2. SIMT (single instruction, multiple threads): 32 threads (a warp) share one program counter and one instruction fetch/decode. Each thread has its own registers, and a per-thread active mask handles if/else.
3. Latency hiding by parallelism: when a warp waits for memory, the scheduler issues from another ready warp the very next cycle. No speculation is needed if there is always other work.

Your reference picture (reused below) shows the standard structure: a GPU is a set of compute units (CU on AMD, SM on NVIDIA) sharing an L2 cache and DRAM. Inside each SM are several warp schedulers, each feeding its own ALUs and register file, plus shared L1/scratchpad storage.

GPU: inside one compute unit

Illustrative RT-capable architecture, exact counts vary by GPU

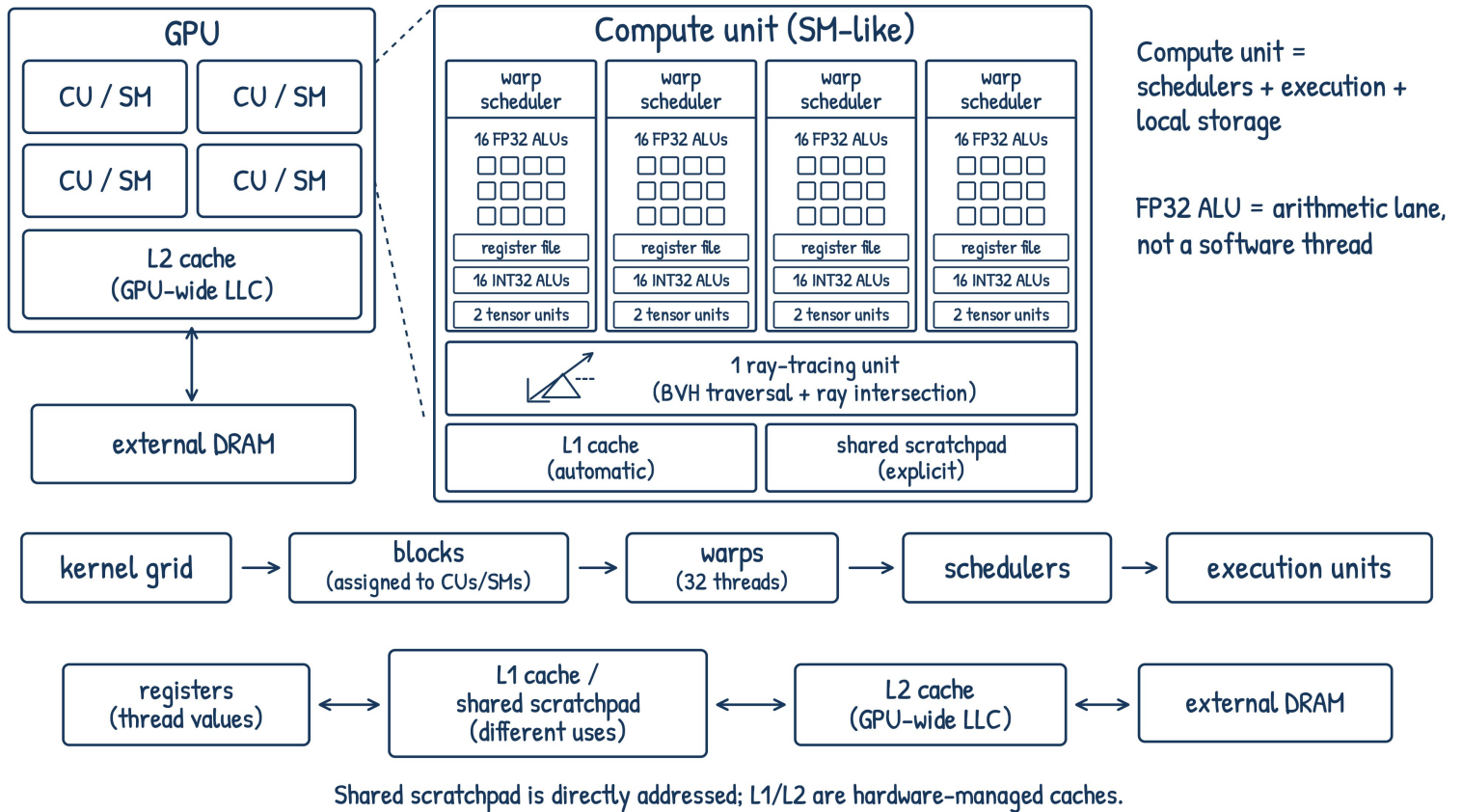
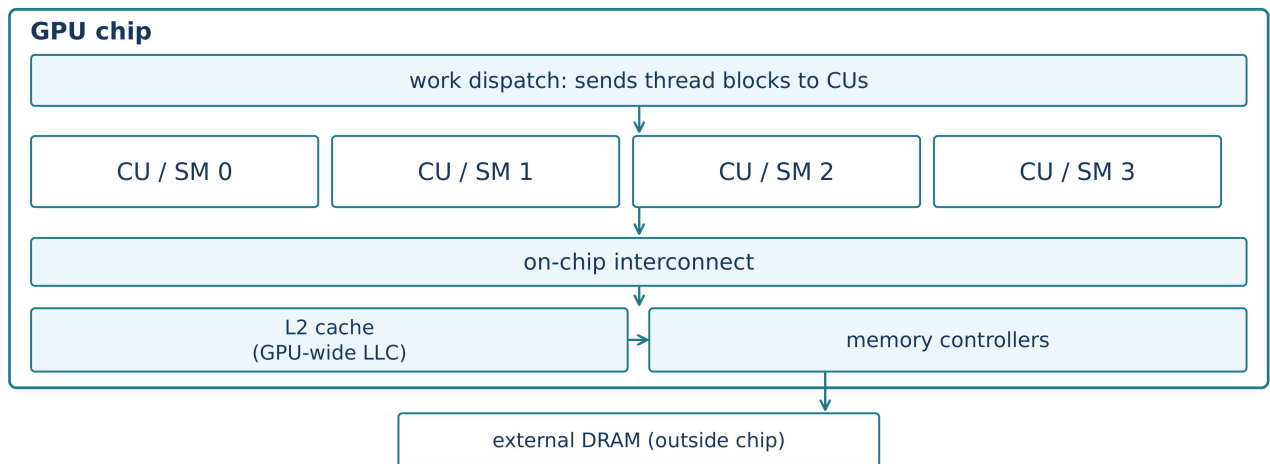


Figure 1. Reference architecture : inside one compute unit and the data path from kernel grid to DRAM.

GPU level: four-CU teaching chip

Additional chip-level blocks only; the reference diagram above is unchanged.



L1 + shared scratchpad are inside each CU; L2 is shared across the four CUs.

Figure 2. Reference: four-CU. ToyGPU-4 follows exactly this top-level shape.

Mapping the reference to ToyGPU-4:

Reference block	ToyGPU-4 choice	Reason
4 CUs/SMs	4 SMs	as requested
4 warp schedulers per SM	2 processing blocks (PB) per SM, 1 scheduler each	halves the RTL to write; parameter NUM_PB can be 4
16 FP32 + 16 INT32 ALUs per scheduler	8 FP32 FMA + 8 INT lanes per PB	warp of 32 executes in 4 beats; same concept, less area
register file	32 KB per PB (64 KB per SM)	enough for 8 warps x 32 regs
2 tensor units	not in v1; stretch milestone M8 (4x4x4 FP16 MMA)	learn the SIMT core first
ray-tracing unit	dropped	graphics-specific, not needed for CUDA compute
L1 cache + shared scratchpad	16 KB L1D + 16 KB shared per SM	separate arrays: simpler than a unified carve-out
L2 (GPU-wide LLC)	256 KB, 4 banks, write-back	point of coherence + global atomics
external DRAM	AXI4 slave memory model in the testbench	front-end only; no physical memory

2. ToyGPU-4 specification at a glance

Parameter (tgpu_pkg)	Default	Notes
NUM_SM	4	streaming multiprocessors
NUM_PB	2	processing blocks per SM, each with its own scheduler
WARP_SIZE	32	CUDA warpSize, visible to software
NUM_LANES	8	SIMD width per PB, so 4 beats per warp instruction
WARPS_PER_PB	8	16 warps and 512 threads per SM
MAX_CTA_PER_SM	4	thread blocks resident per SM
MAX_THREADS_PER_BLOCK	512	reported by cudaGetDeviceProperties
ARCH_REGS	64 (R0..R62 + RZ)	per thread; 7 predicates P0..P6 + PT
RF_WARPREGS_PER_PB	256	256 x 1024 bit = 32 KB per PB
SMEM_BYTES	16 KB/SM, 16 banks x 4 B	explicit __shared__
L1D	16 KB, 4-way, 64 B lines, 8 MSHRs	write-through, no write-allocate
L1I	4 KB, 2-way, 64 B lines	8 instructions per line
Constant cache	2 KB, direct mapped	kernel parameters, __constant__
L2	256 KB, 4 banks x 64 KB, 8-way	write-back, write-allocate, atomics
Interconnect	4 x 4 crossbar, separate request/response	bank = addr[7:6]
Memory port	AXI4 master, 128-bit data	DRAM is a TB model
Host port	AXI4-Lite slave + IRQ	CSRs and doorbell
FP32	IEEE FMA, round-to-nearest-even, flush-to-zero	like nvcc -ftz=true; no FP64 in v1
Clock	single domain gpu_clk	no CDC inside the GPU

What "CUDA compatible" means here: an unmodified .cu file (within a documented subset) compiles with clang or nvcc, the device code is lowered PTX to TGX, and the host code links against our runtime libtgcdart. Supported in v1: 1D/2D/3D grids and blocks, threadIdx/blockIdx/blockDim/gridDim, int32/uint32/float, global/shared/constant memory, __syncthreads, atomicAdd/atomicCAS/atomicExch on 32-bit, divergent control flow, loops. Not in v1: FP64, dynamic parallelism, unified memory, streams beyond the default, textures, cooperative groups beyond block scope, warp shuffles (added in M8).

3. Life of a kernel, end to end

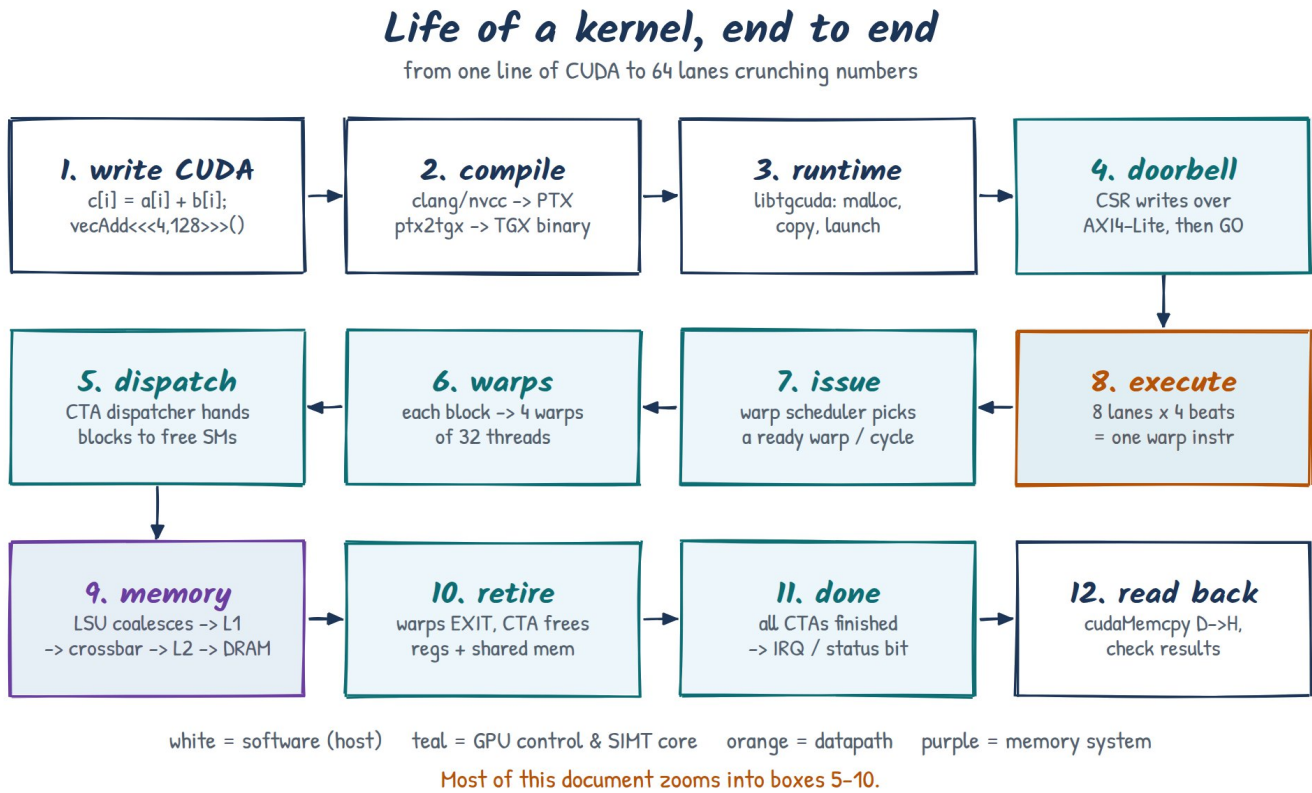


Figure 3. The 12 steps from one line of CUDA to results in memory.

The same flow in terms of hardware blocks: the host writes a launch descriptor into CSRs and rings the doorbell. The command processor latches it; the CTA dispatcher walks every block id and sends each block (CTA) to an SM with free resources. The SM CTA manager creates the warps, the warp schedulers issue their instructions, execution units compute, the LSU moves data through L1, crossbar, L2 and DRAM. When every CTA has retired, STATUS.DONE is set and an IRQ is raised.

Kernel launch sequence

what happens between `vecAdd<<<4,128>>>(a,b,c,n)` and the result

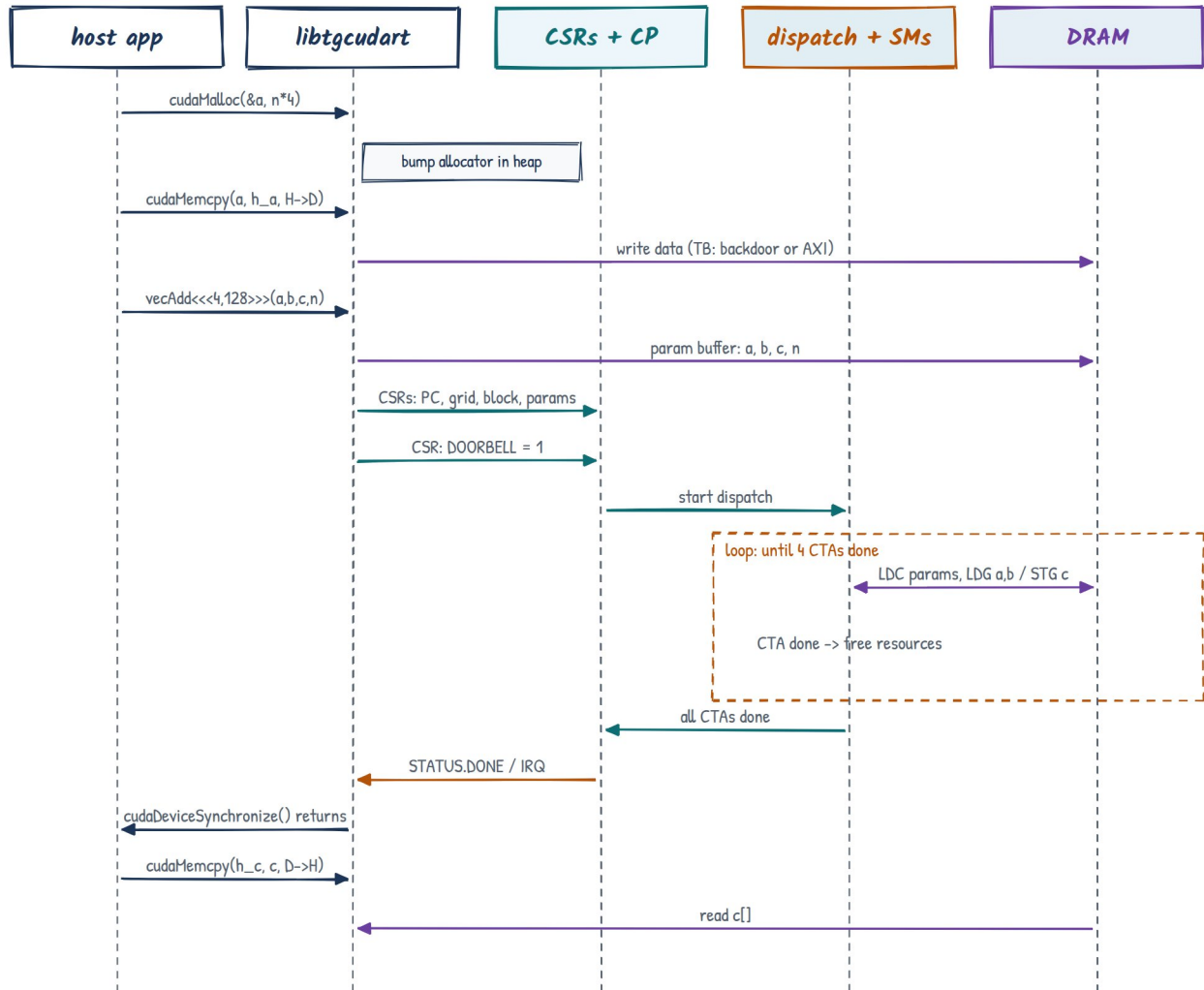


Figure 4. Kernel launch sequence between the host program, runtime, GPU control, SMs and memory.

Software threads vs hardware lanes

what the programmer sees (left) and what the silicon does (right)

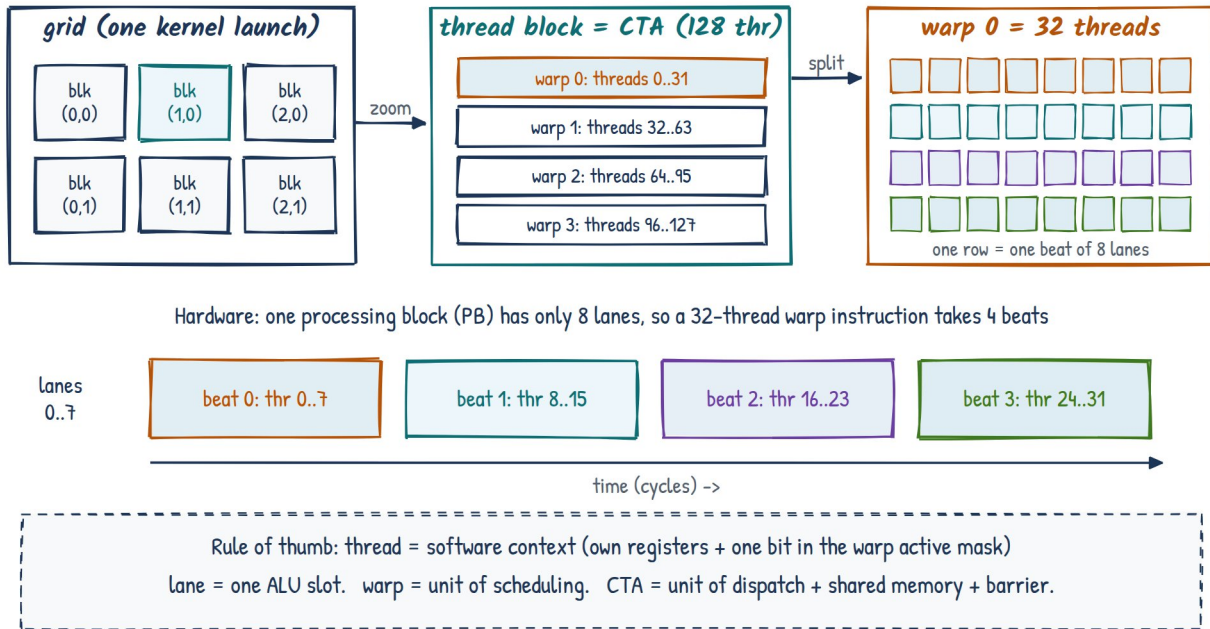


Figure 5. Programming-model hierarchy mapped onto hardware: grid, CTA, warp, beats of 8 lanes.

Key rule of thumb: a thread is a software context (its registers plus one bit in its warp active mask). A lane is one ALU slot. A warp is the unit of scheduling. A CTA is the unit of dispatch, shared memory allocation and barrier.

4. Chip top level (tgpu_top)

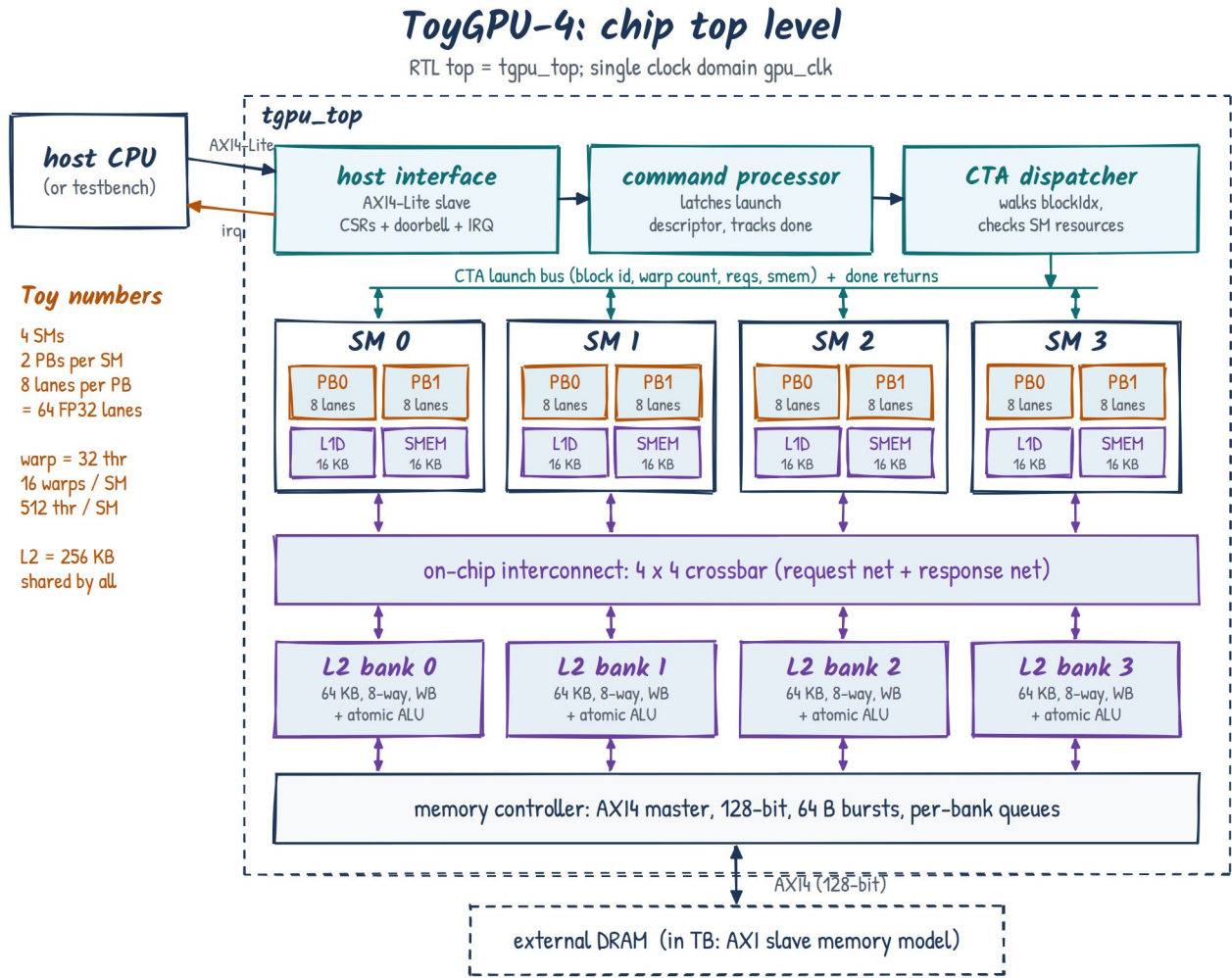


Figure 6. ToyGPU-4 top level: host interface, command processor, CTA dispatcher, 4 SMs, crossbar, 4 L2 banks, memory controller.

Block	Responsibility	Key interfaces
tgpu_hif_axil	AXI4-Lite slave; CSR file; doorbell; IRQ; perf counter readout	AXI4-Lite in; launch descriptor out
tgpu_cp	command processor: holds launch state, busy/done, error flags	descriptor in; start/done with dispatcher
tgpu_cta_dispatch	walks blockIdx, checks each SM resource vector, sends CTA descriptors	CTA launch bus to 4 SMs; done pulses back
tgpu_sm x4	executes CTAs (Section 6)	CTA bus; crossbar request/response
tgpu_xbar	4 x 4 crossbar, round-robin per output, separate req and resp networks	valid/ready streams
tgpu_l2_bank x4	cache + MSHRs + atomic ALU; point of coherence	xbar port; memory controller queue
tgpu_mem_axi	arbitrates L2 banks onto one AXI4 master, 64 B bursts (4 x 128 bit)	AXI4 master to TB DRAM model

4.1 CSR map (AXI4-Lite, 32-bit registers)

Offset	Name	Description
0x000	ID	magic + version (read only)
0x004	CTRL	bit0 GO (doorbell, self clearing), bit1 SOFT_RESET, bit2 IRQ_EN
0x008	STATUS	bit0 BUSY, bit1 DONE (write 1 to clear), bits 15:8 error code
0x010 / 0x014	KERNEL_PC_LO / HI	byte address of the first instruction
0x018 / 0x01C	PARAM_PTR_LO / HI	address of the packed kernel parameter buffer
0x020..0x028	GRID_X / Y / Z	gridDim
0x02C..0x034	BLOCK_X / Y / Z	blockDim (product must be <= 512)
0x038	REGS_PER_THREAD	rounded up to a multiple of 4 by the runtime
0x03C	SMEM_PER_CTA	bytes, static + dynamic shared memory
0x040	PARAM_BYTES	size of the parameter buffer (<= 2 KB)
0x100..0x1FC	PERF_*	cycles, instructions issued, stall reasons, cache hits/misses

5. CTA dispatcher and occupancy

CTA dispatcher: handing blocks to SMs

$\text{tgpu_cta_dispatch} + \text{tgpu_cta_mgr}$ (one per SM)

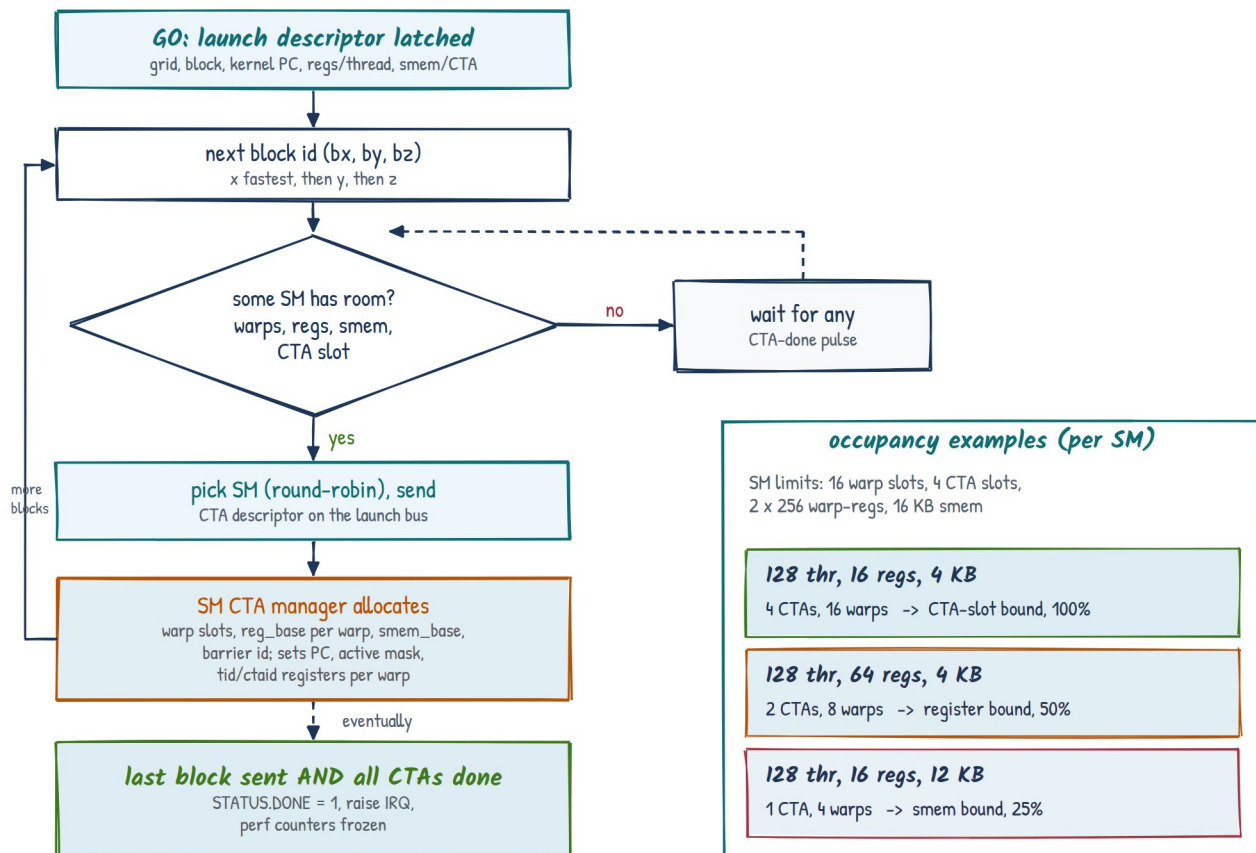


Figure 7. CTA dispatch flow and three occupancy examples.

Each SM exports a resource vector: free warp slots per PB, free warp-registers per PB, free shared memory, free CTA slots. The dispatcher checks "fits" for the next block, picks an SM round-robin, and sends a descriptor {ctaid, warp count, reg_base per PB, smem_base, barrier id}. Allocation of registers and shared memory is a simple first-fit bitmap in the SM CTA manager (tgpu_cta_mgr). Warps of a CTA are spread across PBs as warp i goes to PB (i mod 2), which balances load.

Design notes: dispatch rate of one CTA per 4 cycles is plenty. Initialization of special registers (tid.x/y/z per thread, ctaid, ntid, nctaid) is done by the CTA manager writing them into a small special-register table read by S2R instructions, not by writing the RF, which keeps launch fast.

6. Inside one SM (tgpu_sm)

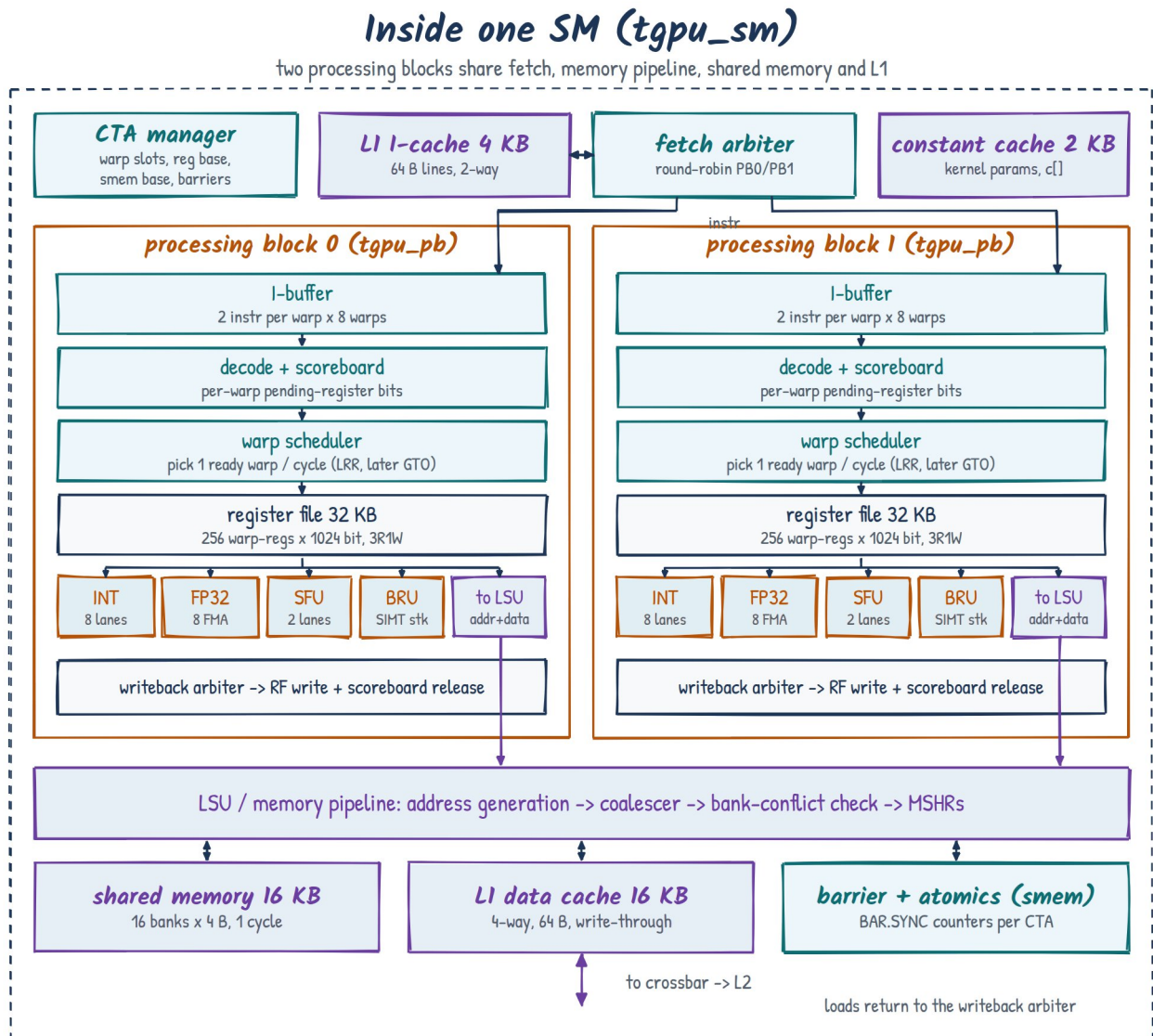
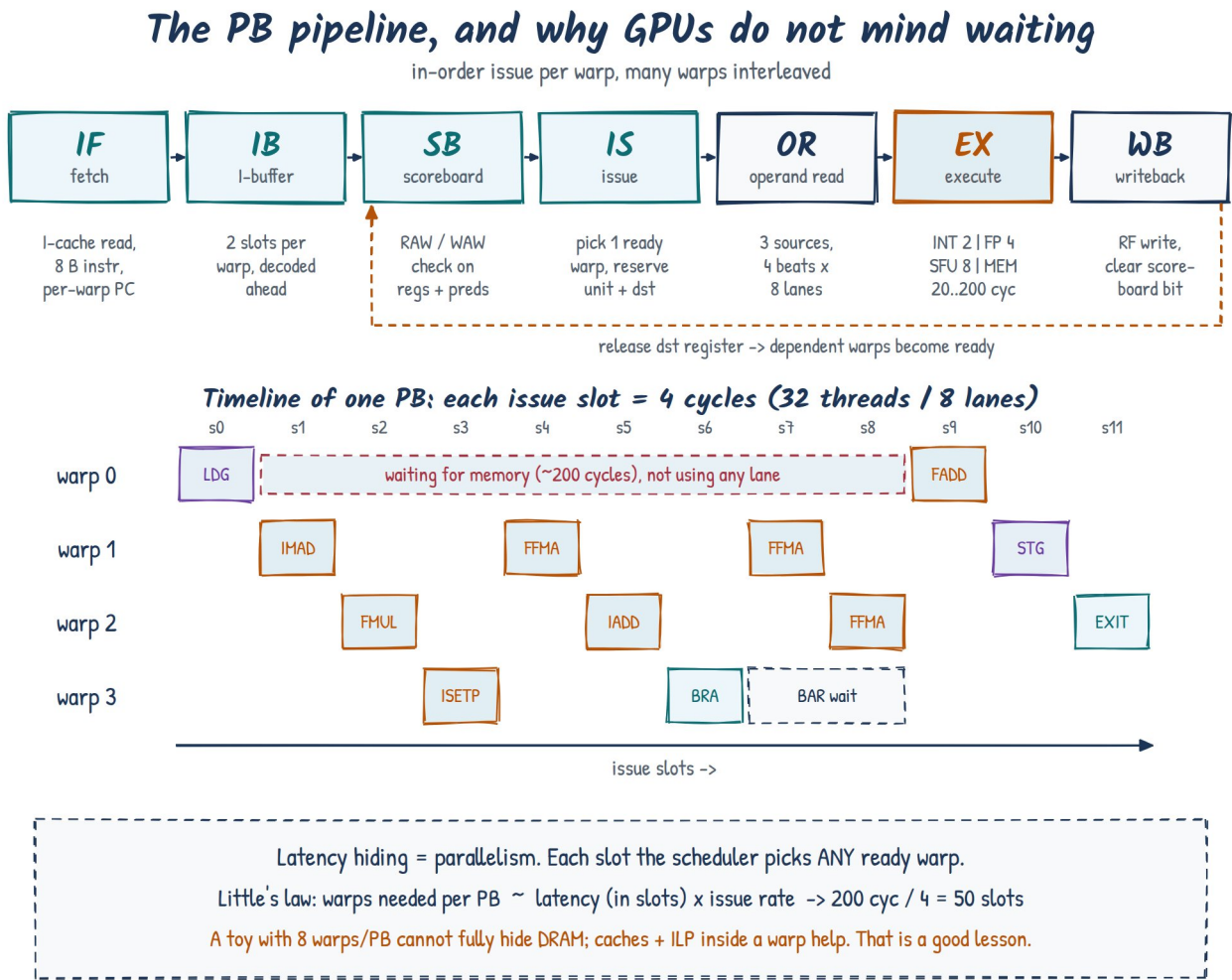


Figure 8. SM microarchitecture: shared front end and memory pipeline, two processing blocks.

Shared by both PBs: CTA manager, 4 KB I-cache with a round-robin fetch arbiter, constant cache, the LSU/memory pipeline, 16 KB shared memory, 16 KB L1D and the barrier unit. Private to each PB: I-buffer, decode, scoreboard, warp scheduler, SIMT stacks, register file, execution units and writeback.

6.1 Pipeline



6.2 Warp scheduler and scoreboard

Warp scheduler, scoreboard and l-buffer

the heart of every PB: who is allowed to issue this cycle?

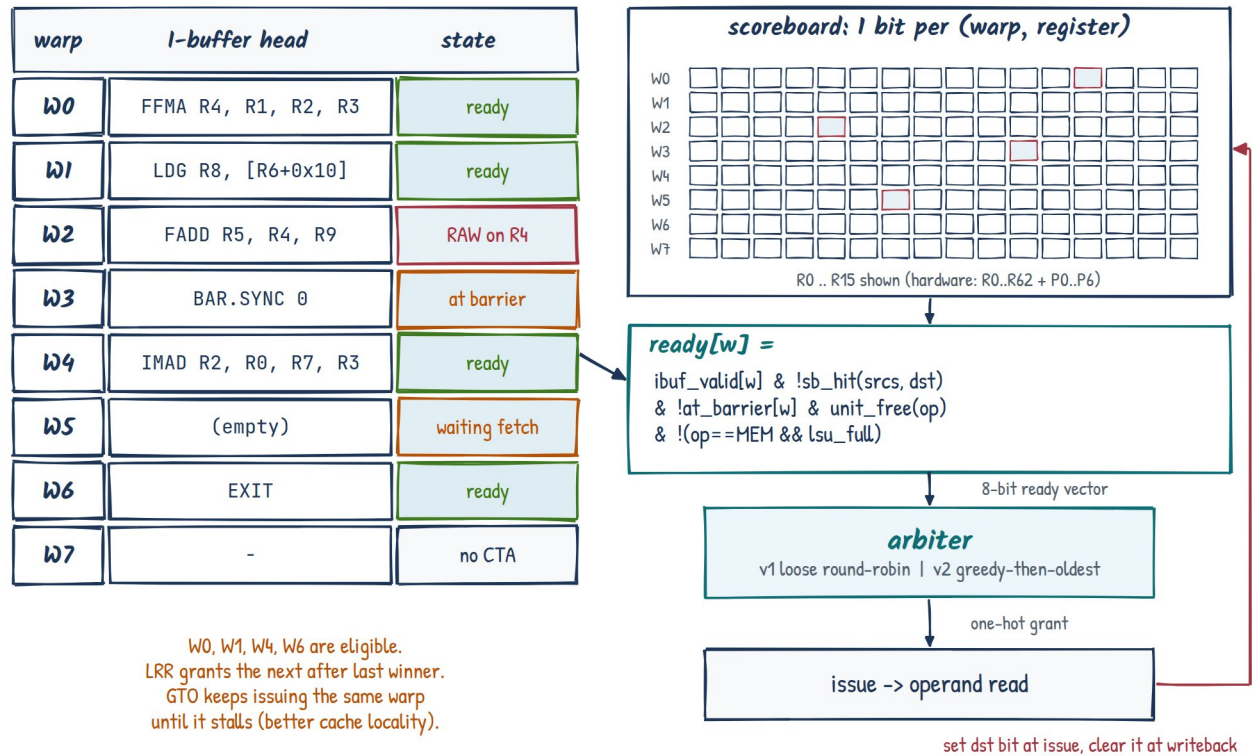


Figure 10. Per-warp state, scoreboard bits, ready logic and arbiter.

ready[w] = ibuf_valid & !scoreboard_hit & !at_barrier & !waiting_membar & unit_free(op). v1 arbiter is loose round-robin (LRR). v2 adds greedy-then-oldest (GTO), which keeps issuing the same warp until it stalls and usually improves cache locality. Keep the policy in a separate module (tgpu_warp_sched) with the same ports so the swap is trivial and measurable with perf counters.

6.3 Branch divergence: SIMT stack

Branch divergence with a SIMT reconvergence stack

toy picture uses an 8-thread warp; hardware is identical with 32

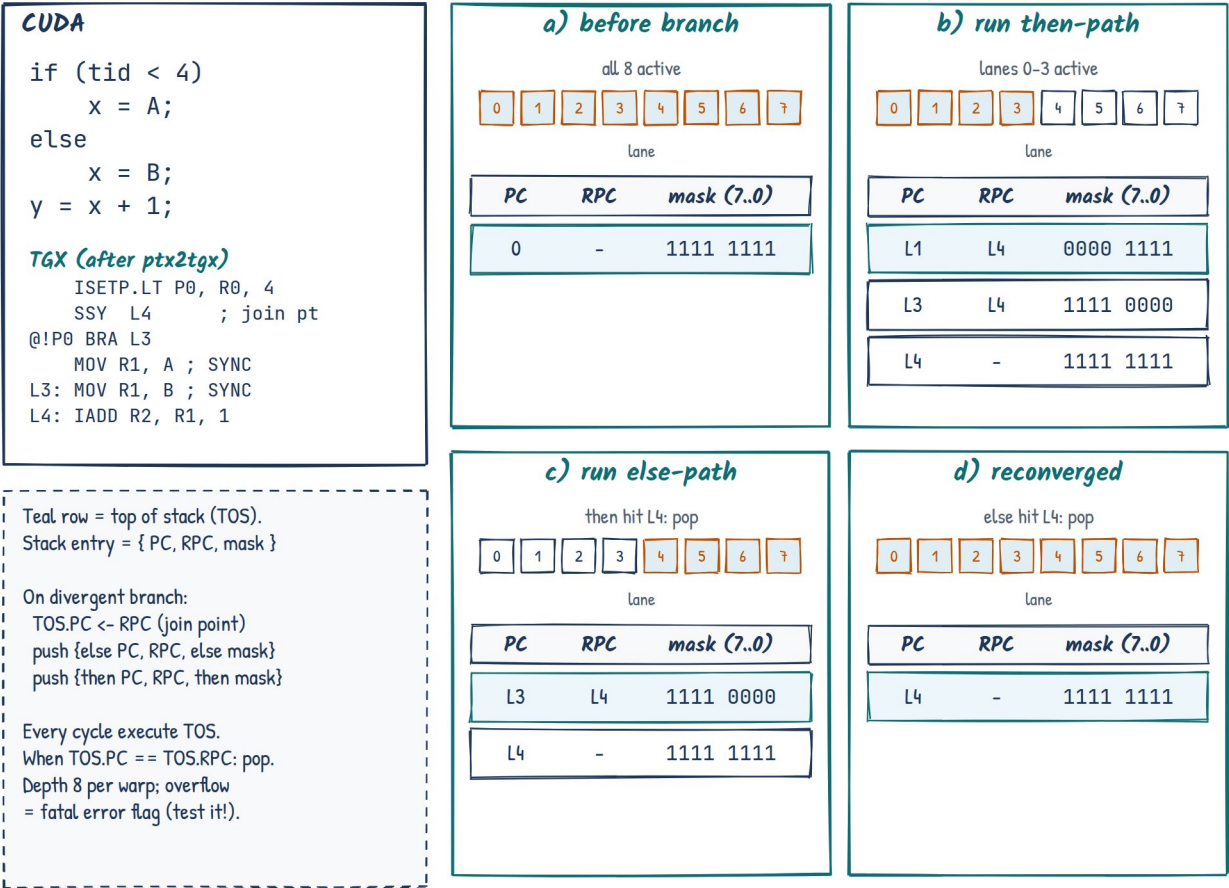


Figure 11. IPDOM reconvergence stack walk-through with an if/else.

Each warp has an 8-entry stack of {PC, RPC, mask}. SSY pushes the reconvergence point computed by the compiler (the immediate post-dominator of the branch). A divergent BRA replaces TOS.PC with RPC and pushes the else path and then the then path. When TOS.PC equals TOS.RPC the entry is popped. Uniform branches (all active lanes agree) do not touch the stack. EXIT clears lanes from every entry; the warp retires when the mask is all zero. Loops with break work the same way, the break target being the loop exit post-dominator.

Alternative (discussion): Volta and later use per-thread PCs with convergence barriers, which allows independent thread scheduling and fixes deadlocks in spin locks within a warp. It is more complex; the IPDOM stack is the right first design and is what GPGPU-Sim models (https://gpgpu-sim.org/manual/index.php/Main_Page).

6.4 Register file

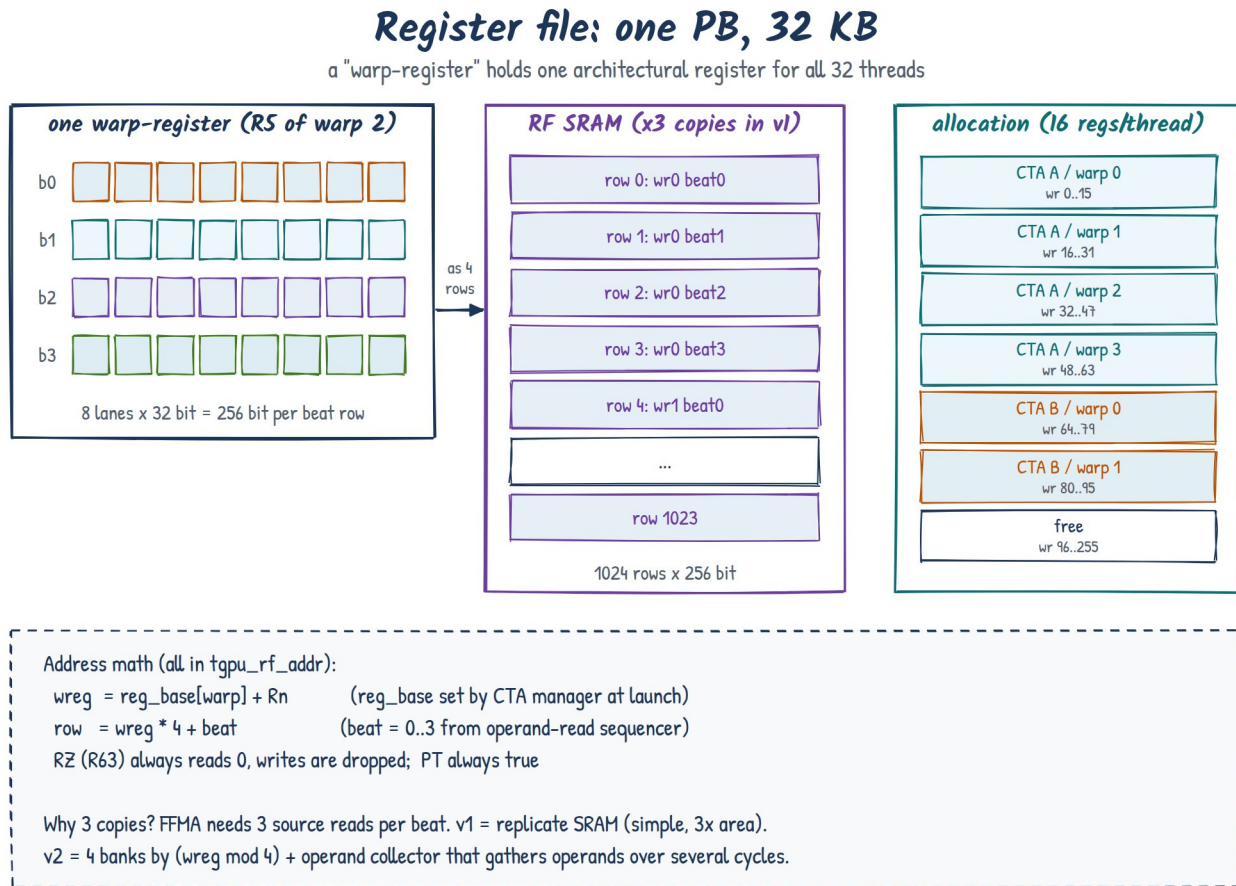


Figure 12. Register file organization, allocation and address calculation.

One warp-register is one architectural register for all 32 threads (1024 bits), stored as 4 rows of 256 bits (one row per beat). v1 uses 3 replicated 1R1W SRAM copies so FFMA can read 3 operands every beat with no conflicts. v2 replaces this with 4 single-ported banks selected by $(warp\text{-}register \bmod 4)$ and an operand collector that gathers operands over several cycles, as described in the GPGPU-Sim manual. Always wrap SRAMs in `lib_sram_1r1w` so a real SRAM macro can be substituted later.

6.5 Execution units

Unit	Lanes	Latency	Operations
<code>tgpu_alu_int</code>	8	2	IADD, ISUB, IMUL.LO/HI, IMAD, SHL, SHR, AND/OR/XOR/NOT, MIN/MAX, ISETP, SEL, MOV, POPC, CLZ
<code>tgpu_fpu</code>	8	4	FADD, FMUL, FFMA, FMIN/FMAX, FSETP, I2F, F2I; RNE, FTZ
<code>tgpu_sfu</code>	2	8 (+ 4 beats x 4)	RCP, RSQ, SQRT, EX2, LG2, SIN, COS (approximate, table + quadratic)
<code>tgpu_bru</code>	-	1	BRA, SSY, SYNC, EXIT, BAR.SYNC, MEMBAR; computes taken mask and updates the SIMT stack
<code>tgpu_lsu (shared)</code>	8 AGU	3 to 300	LDG/STG, LDS/STS, LDC, ATOMG, ATOMS

FP32 advice: writing a correct IEEE FMA from scratch is a great exercise but a big one. Option A: write your own with FTZ and RNE only, and verify against Berkeley SoftFloat (<https://github.com/ucb-bar/berkeley-softfloat-3>). Option B: integrate the open-source SystemVerilog FPnew/CVFPU (<https://github.com/openhwgroup/cvfpu>) and focus on the GPU parts.

7. Memory pipeline inside the SM

LSU: coalescing (global) and bank conflicts (shared)

same idea twice: turn many lane addresses into as few memory transactions as possible

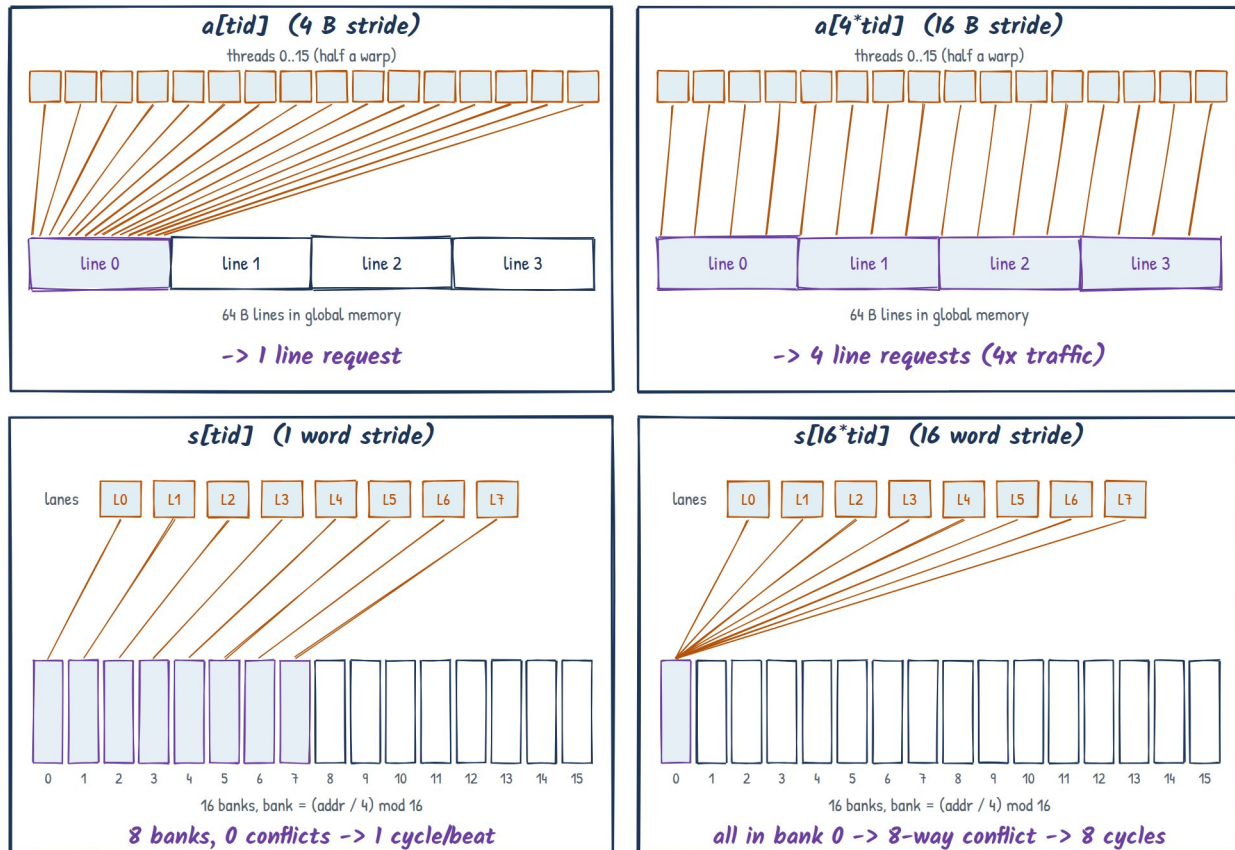


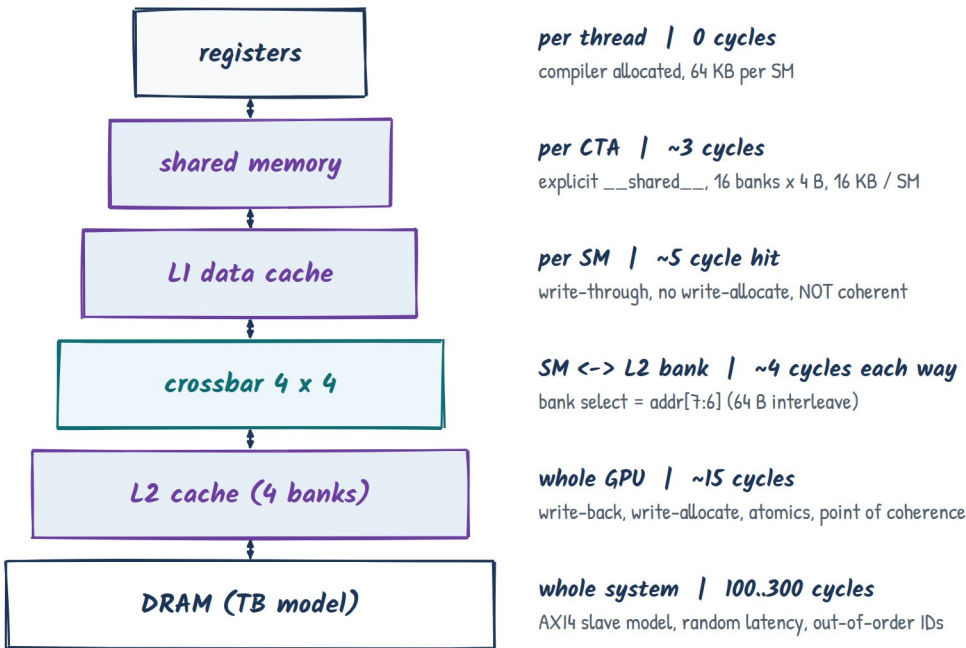
Figure 13. Global coalescing and shared memory bank conflicts.

- Address generation: 8 lanes per beat compute base + offset.
- Coalescer (global): collect all 32 addresses of the warp (4 beats), group active lanes by 64 B line, emit one request per unique line. Unit stride 4 B: 2 requests per warp. Random: up to 32.
- Shared memory: bank = (addr / 4) mod 16. The conflict checker finds the max number of distinct words per bank and serializes. Same word in the same bank is a broadcast, not a conflict.
- L1D: 4-way, 64 B lines, write-through with no write-allocate (stores invalidate the line and go to L2). 8 MSHRs; a second miss to the same line merges into the existing MSHR.
- Loads write back through the PB writeback arbiter and clear the scoreboard only when all 4 beats of data are present.
- Atomics on shared memory run in the SM (read-modify-write in the bank, serialized by address). Global atomics bypass L1 and run in the L2 bank ALU.

8. Interconnect, L2 and memory model

Memory hierarchy and policies

toy latencies are testbench knobs; policies are fixed by the RTL



Memory model (weak, like CUDA): a store from one SM may be seen late by another SM.

`__syncthreads()` = `BAR.SYNC` (all warps of the CTA arrive; shared memory writes visible)

`__threadfence()` = `MEMBAR.GL` (stall the warp until all of its stores are acked by L2)

`atomicAdd(global)` runs in the L2 bank ALU; `atomicAdd(shared)` runs in the SM smem unit.

Figure 14. Memory hierarchy, scope, latency and policies, plus the memory model.

Crossbar: each of the 4 SMs has a request port; each L2 bank has one input with a round-robin arbiter. Responses use a separate network (avoids request/response deadlock). Bank select is `addr[7:6]`, so consecutive 64 B lines go to different banks.

L2 bank: 64 KB, 8-way, pseudo-LRU, write-back, write-allocate, per-bank MSHRs, byte masks on writes, an atomic ALU (ADD, MIN, MAX, AND, OR, XOR, EXCH, CAS) placed next to the data array. L2 is the single point of coherence: L1 is not coherent, so the memory model is weak like CUDA: `__syncthreads` maps to `BAR.SYNC`, `__threadfence` to `MEMBAR.GL` (warp stalls until all its outstanding stores are acked by L2), and volatile or `ld.cg` loads bypass L1.

Memory controller: a queue per bank feeding one AXI4 master (128-bit, 4-beat bursts). Use distinct AXI IDs per bank so the DRAM model can return responses out of order across banks.

9. Computer architecture discussion: design choices and alternatives

Decision	v1 choice	Alternative	Trade-off to study
Warp size	32	8 or 16	32 keeps CUDA semantics (<code>warpSize</code> , <code>ballot</code>) exact
SIMD width	8 lanes x 4 beats	32 lanes x 1 beat	narrow = less area, same programming model,

			longer occupancy per instr
Scheduler	LRR	GTO, two-level	GTO improves locality; measure with perf counters
Divergence	IPDOM SIMT stack	per-thread PC + convergence barriers	stack is simpler; ITS avoids intra-warp deadlock
RF ports	3 replicated copies	banked + operand collector	area versus bank conflicts
L1 policy	write-through, no allocate	write-back	write-through makes coherence trivial at L2
Shared vs L1	separate arrays	unified configurable	unified saves area, adds muxing
Dependency check	scoreboard	compiler-inserted stall counts (NVIDIA style)	moves complexity to ptx2txg
ISA	custom TGX, 64-bit fixed	RISC-V + extensions (Vortex)	custom maps PTX 1:1; see Vortex (https://github.com/vortexgpgpu/vortex)

Latency hiding sanity check (Little's law): with 200-cycle DRAM and one issue slot every 4 cycles, a PB needs about 50 independent instructions in flight to hide memory fully. With 8 warps per PB that requires caches and instruction-level parallelism inside each warp. Measuring this gap is one of the most instructive experiments this toy allows.

10. TGX instruction set

TGX instruction encoding: fixed 64 bits

fixed width = trivial fetch and decode; 8 instructions per 64 B l-cache line

R-form (register): FFMA R4, R1, R2, R3



I-form (immediate): IADD R2, R1, 0x10 | LDG R3, [R6 + 0x10] | BRA +off



opcode[7:6] = unit class 00 INT | 01 FP / SFU | 10 MEM | 11 CTRL
 opcode[5] = I-form flag, so the decoder knows the layout from the first byte
 pg = guard predicate: bit 3 = negate, bits 2:0 = P0..P6, 7 = PT (always) @!P0 -> pg = 1000
 pd = destination predicate for ISETP / FSETP
 mod = compare op (.LT .EQ ...), type (.U32 .S32), .FTZ, .SAT, memory width (.32 .64 .128)
 registers = 6 bits: R0..R62, R63 = RZ (reads 0)

Keep the ISA regular: every field is in the same place in every instruction.

Figure 15. TGX 64-bit instruction formats and field meanings.

Class	Instructions
INT (00)	IADD ISUB IMUL IMAD SHL SHR AND OR XOR NOT MIN MAX ISETP SEL MOV POPC CLZ
FP/SFU (01)	FADD FMUL FFMA FMIN FMAX FSETP I2F F2I RCP RSQ SQRT EX2 LG2 SIN COS
MEM (10)	LDG STG LDS STS LDC ATOMG ATOMS (with .32 .64 .128 widths)

CTRL (11)	BRA SSSY SYNC EXIT BAR.SYNC MEMBAR.CTA MEMBAR.GL S2R NOP (M8: VOTE SHFL)
-----------	--

10.1 Worked example: vectorAdd

```
// CUDA
__global__ void vecAdd(const float* a, const float* b, float* c, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) c[i] = a[i] + b[i];
}

// PTX (abridged)
mov.u32 %r1, %ctaid.x;  mov.u32 %r2, %ntid.x;  mov.u32 %r3, %tid.x;
mad.lo.s32 %r4, %r1, %r2, %r3;
setp.ge.s32 %p1, %r4, %r5;  @%p1 bra DONE;
mul.wide.s32 %rd4, %r4, 4;  add.s64 %rd5, %rd1, %rd4;  ld.global.f32 %f1, [%rd5]; ...

// TGX (after ptx2tgx; 32-bit addresses in v1)
S2R    R0, SR_CTAID.X
S2R    R1, SR_NTID.X
S2R    R2, SR_TID.X
IMAD   R3, R0, R1, R2      ; i
LDC    R4, c[0x0c]         ; n
ISETP.GE P0, R3, R4
@P0 EXIT
LDC    R5, c[0x00]         ; a
LDC    R6, c[0x04]         ; b
LDC    R7, c[0x08]         ; c
SHL    R8, R3, 2
IADD   R5, R5, R8
IADD   R6, R6, R8
IADD   R7, R7, R8
LDG.32 R9, [R5]
LDG.32 R10, [R6]
FADD   R11, R9, R10
STG.32 [R7], R11
EXIT
```

11. Software path: making it CUDA compatible

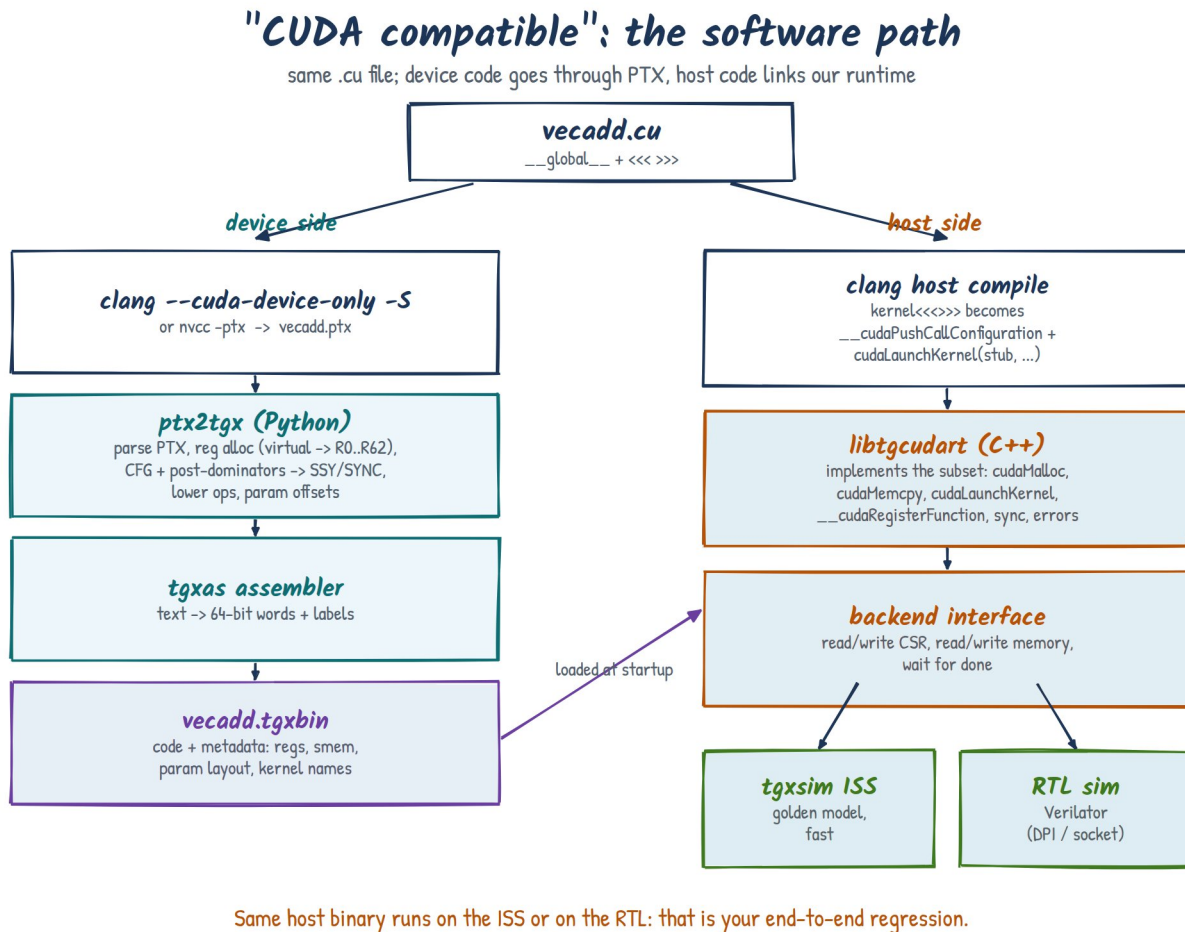


Figure 16. Device path (PTX to TGX) and host path (runtime shim) from the same .cu file.

- Device: `clang --cuda-device-only -S` (or `nvcc -ptx`) produces PTX, documented at <https://docs.nvidia.com/cuda/parallel-thread-execution/>. `ptx2tgx` (Python) parses PTX, performs linear-scan register allocation from PTX virtual registers to R0..R62, builds the CFG, computes post-dominators to place SSY/SYNC, lowers each PTX op and computes the parameter layout.
- Host: `clang` lowers `kernel<<<g,b>>>(args)` into `__cudaPushCallConfiguration` followed by a call to a stub that invokes `cudaLaunchKernel` (see <https://llvm.org/docs/CompileCudaWithLLVM.html>). `libtgcudart` implements these symbols plus `__cudaRegisterFatBinary/``__cudaRegisterFunction`, `cudaMalloc`, `cudaMemcpy`, `cudaMemset`, `cudaDeviceSynchronize`, `cudaGetLastError` and `cudaGetDeviceProperties`.
- Backend interface: four calls (`csr_write`, `csr_read`, `mem_write`, `mem_read`, plus `wait_done`). Two backends: `tgxsim` (the ISS) and the Verilator RTL model. The same host executable is the end-to-end regression.
- `tgxbn` format: header, kernel table {name, entry PC, regs, smem, param bytes}, code section, constant section.

12. Front-end design workflow

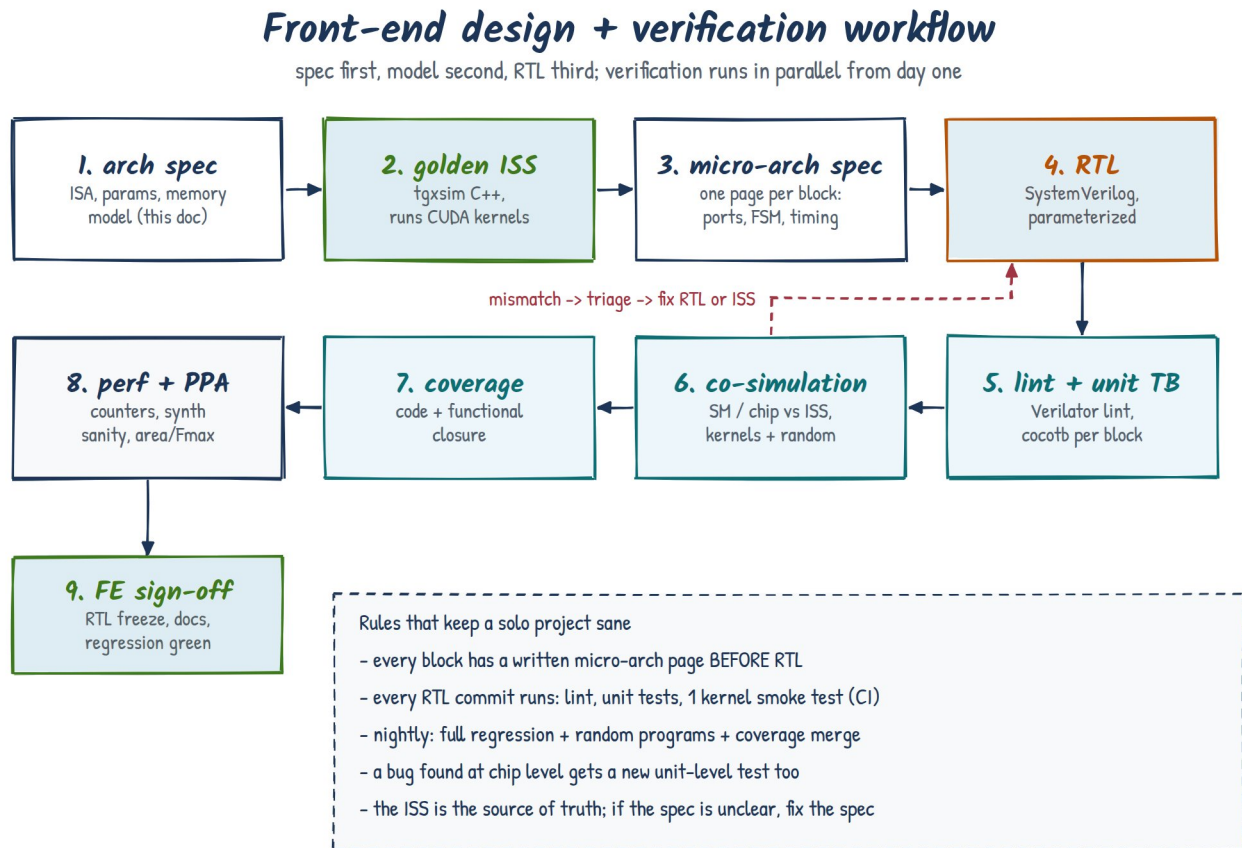


Figure 17. Front-end design and verification workflow with the mismatch loop.

Front-end implementation roadmap

each milestone ends with a demo that runs a real kernel

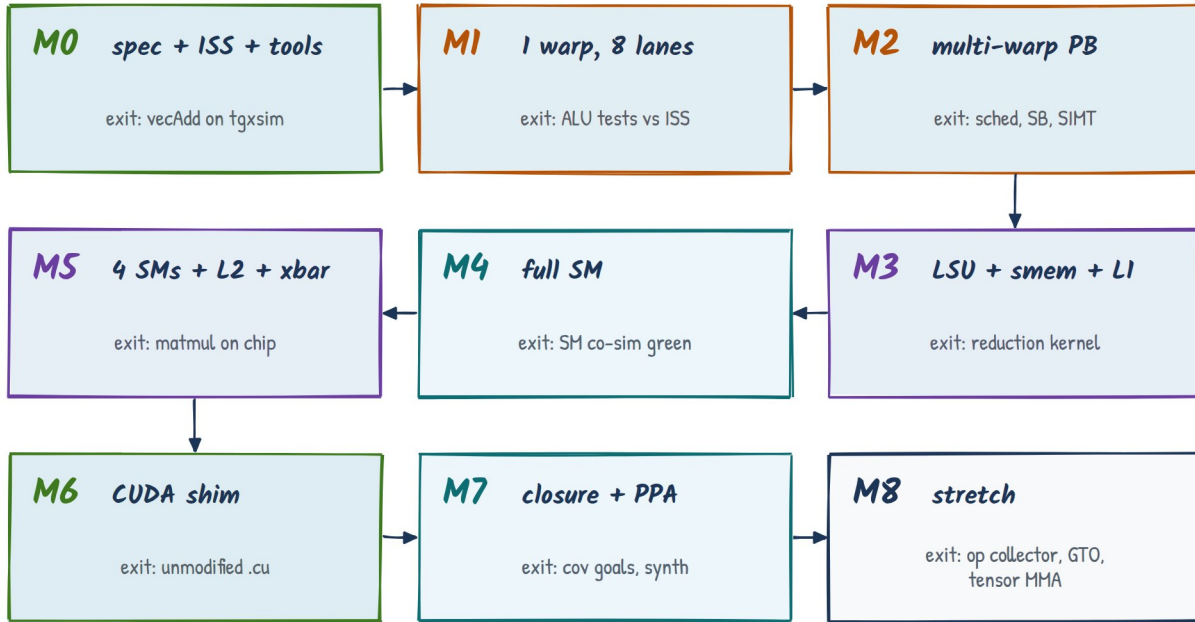


Figure 18. Implementation roadmap: milestones M0 to M8 with exit criteria.

Milestone	Content	Exit criteria
M0	this spec, tgxsim ISS, ptx2tgx, tgxas, libtgcudart on ISS	vecAdd, SAXPY, reduction pass on the ISS
M1	single-warp PB: fetch from TB ROM, decode, RF, INT + FP units	random ALU programs match ISS commit by commit
M2	8 warps, scoreboard, LRR scheduler, SIMT stack, BRU	divergence and nested loop tests; no deadlock over 1M random cycles
M3	LSU, coalescer, shared memory, barrier, L1D with a memory stub	shared-memory reduction and transpose kernels pass
M4	full SM with CTA manager, l-cache, constant cache	SM-level co-sim green across the kernel suite
M5	4 SMs, dispatcher, crossbar, L2 banks, memory controller, HIF	tilled matmul and histogram (atomics) on the full chip
M6	runtime backend on RTL	unmodified CUDA samples from the suite pass on RTL
M7	coverage closure, perf counters, synthesis sanity (Yosys)	coverage goals met; no latches, no loops; area report
M8	stretch: operand collector, GTO, VOTE/SHFL, FP16 tensor MMA	measured speedup versus v1 on the suite

12.1 Repository layout

```

toygpu/
  docs/          spec (this document), one micro-arch page per block
  rtl/lib/       tgpu_pkg.sv, lib_fifo, lib_skid, lib_rr_arb, lib_sram_1r1w ...
  rtl/core/     tgpu_pb and everything inside it
  rtl/sm/       tgpu_sm, cta_mgr, icache, lsu, smem, l1d, ccache, barrier
  rtl/top/      tgpu_top, hif_axil, cp, cta_dispatch, xbar, l2_bank, mem_axi
  rtl/bind/     SVA checker modules + bind files
  sw/iss/       tgxsim (C++), DPI wrapper
  
```

sw/compiler/	ptx2tgx, tgas
sw/runtime/	libtgcudart + backends
tests/kernels/	.cu test kernels with expected results
tb/unit/	cocotb testbench per module
tb/sm/, tb/top/	cluster and chip environments
scripts/	Makefiles, regression runner, coverage merge, CI

13. SystemVerilog HDL guide

RTL module hierarchy

names used throughout this plan (rtl/ directory)

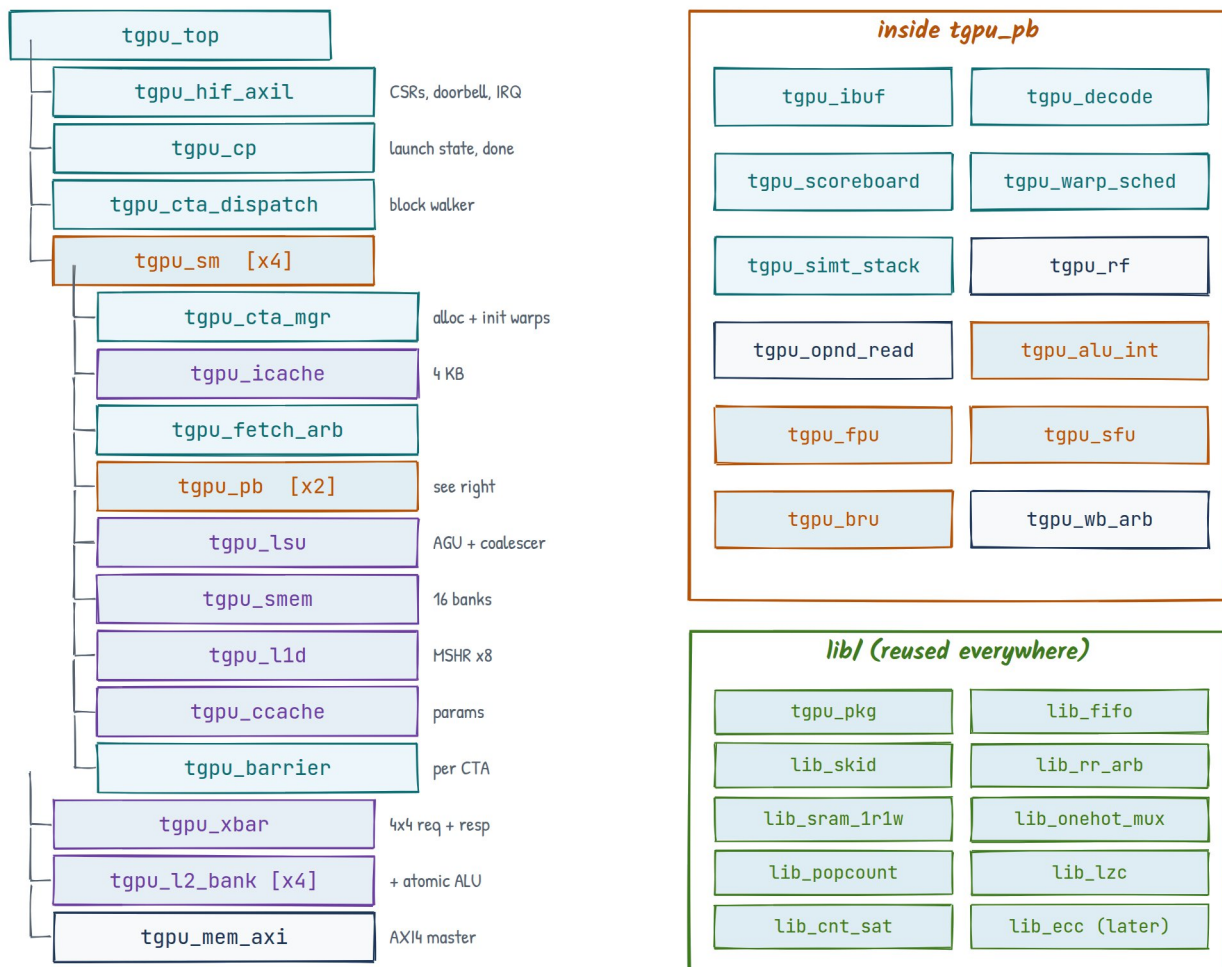


Figure 19. RTL module hierarchy and reusable library.

13.1 Coding conventions

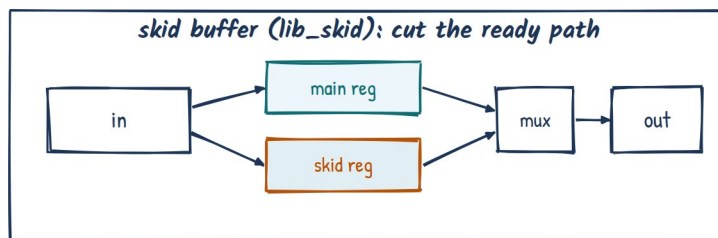
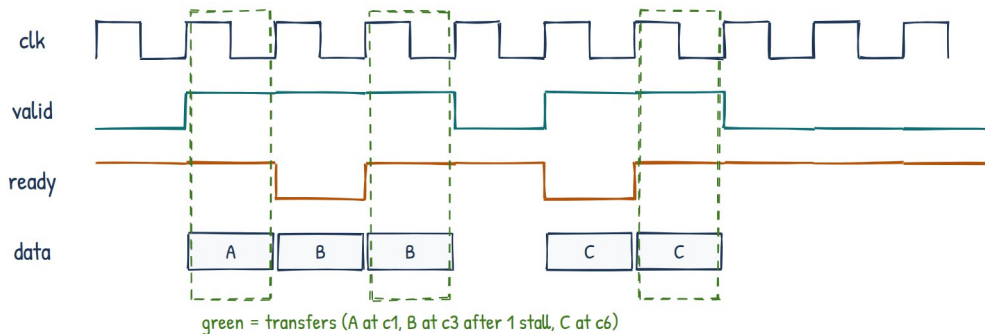
- One module per file, file name = module name. Prefix **tgpu_** for GPU blocks, **lib_** for generic blocks.
- Ports: suffix **_i** and **_o**. Flops: **_q** for the register, **_d** for its next value. Active-low signals end in **_n**.
- Use **always_ff** for flops only and **always_comb** for combinational logic only. Never **always @***. Default-assign every output at the top of **always_comb** so no latches can form.

- Reset: synchronous, active-high `rst_i`, applied only to control state (valid bits, FSM state, counters, pointers). Datapath registers are not reset; this cuts area and makes X-propagation tests meaningful.
- All parameters and types live in `tgpu_pkg`: localparams derived from the top-level parameters, typedef struct packed for instructions, uops, memory requests and responses, typedef enum logic for opcodes and FSM states.
- Every block-to-block link is a valid/ready stream carrying one packed struct. No other handshake styles.
- Use generate for loops over SMs, PBs, lanes and banks. No hierarchical references in RTL (only in testbenches and bind files).
- unique case / priority case only when the property is truly guaranteed; add an assertion for it.
- No initial blocks or delays in synthesizable RTL. No inferred memories outside `lib_sram_*` wrappers.
- Lint clean with Verilator `--lint-only -Wall` (<https://verilator.org/guide/latest/overview.html>) on every commit; waivers are listed in a single file with a reason.

13.2 The universal handshake

The one handshake used everywhere: valid / ready

every block-to-block link in ToyGPU-4 is a valid/ready stream



rules (assert them!)

1. valid never waits for ready
2. once valid, hold data stable until the transfer
3. ready may depend on valid
4. no combinational loop
valid → ready → valid

Figure 20. valid/ready protocol, example waveform, skid buffer and the rules to assert.

13.3 Package example (style reference only)

```
package tgpu_pkg;
  parameter int NUM_SM = 4, NUM_PB = 2, WARP_SIZE = 32, NUM_LANES = 8;
  parameter int WARPS_PER_PB = 8;
  localparam int BEATS = WARP_SIZE / NUM_LANES;          // 4
  localparam int WID_W = $clog2(WARPS_PER_PB);          // 3
  localparam int REG_W = 6;
```

```

typedef enum logic [1:0] {CLS_INT, CLS_FP, CLS_MEM, CLS_CTRL} op_class_e;

typedef struct packed {
    logic [7:0]      opcode;
    logic [3:0]      pg;          // guard predicate
    logic [REG_W-1:0] rd, ra, rb, rc;
    logic [7:0]      mods;
    logic [2:0]      pd;
    logic [31:0]     imm;
    logic            is_imm;
} instr_t;

typedef struct packed {
    logic [WID_W-1:0] wid;
    logic [31:0]      pc;
    logic [WARP_SIZE-1:0] mask;
    instr_t           ins;
} issue_t;
endpackage

```

13.4 Micro-architecture page template (write one per block before RTL)

4. Purpose and position in the hierarchy (one paragraph and a small diagram).
5. Parameters and derived constants.
6. Port table: name, direction, width/type, protocol, description.
7. Internal structure: registers, memories (size, ports), FSMs with state diagrams.
8. Timing: latency, throughput, which outputs are registered, the critical path you expect.
9. Corner cases and error handling (full, empty, back-pressure, reset in the middle of traffic).
10. Assertions to bind and coverage points to collect.
11. Test plan summary for the unit testbench.

13.5 Block interface summary

Module	Inputs	Outputs	Notes
tgpu_ibuf	fetch resp stream, issue pop per warp, flush per warp	head instruction + valid per warp	flush on taken branch or divergence
tgpu_scoreboard	issue (wid, rd, pd), WB release (wid, rd), check query	hit per warp	bit array WARPS x 64 + preds
tgpu_warp_sched	ready vector, unit busy	one-hot grant	policy swappable
tgpu_simt_stack	branch result (taken mask, target), SSY, SYNC, EXIT	PC, active mask per warp	8 entries, overflow flag
tgpu_rf	3 read ports (row), 1 write port (row, data, lane mask)	3 x 256-bit read data	lane-masked writes
tgpu_opnd_read	issue stream	uop stream per beat to units	beat sequencer, imm/RZ mux
tgpu_lsu	mem uops (4 beats), smem/L1 responses	smem/L1 requests, WB data	coalescer, conflict check
tgpu_l1d	LSU req stream, xbar resp	LSU resp, xbar req	8 MSHRs, write-through
tgpu_l2_bank	xbar req, mem resp	xbar resp, mem req	atomics, WB, 8-way
tgpu_cta_mgr	CTA descriptor, warp exit events	resource vector, warp init, CTA done	first-fit bitmaps

14. Verification plan

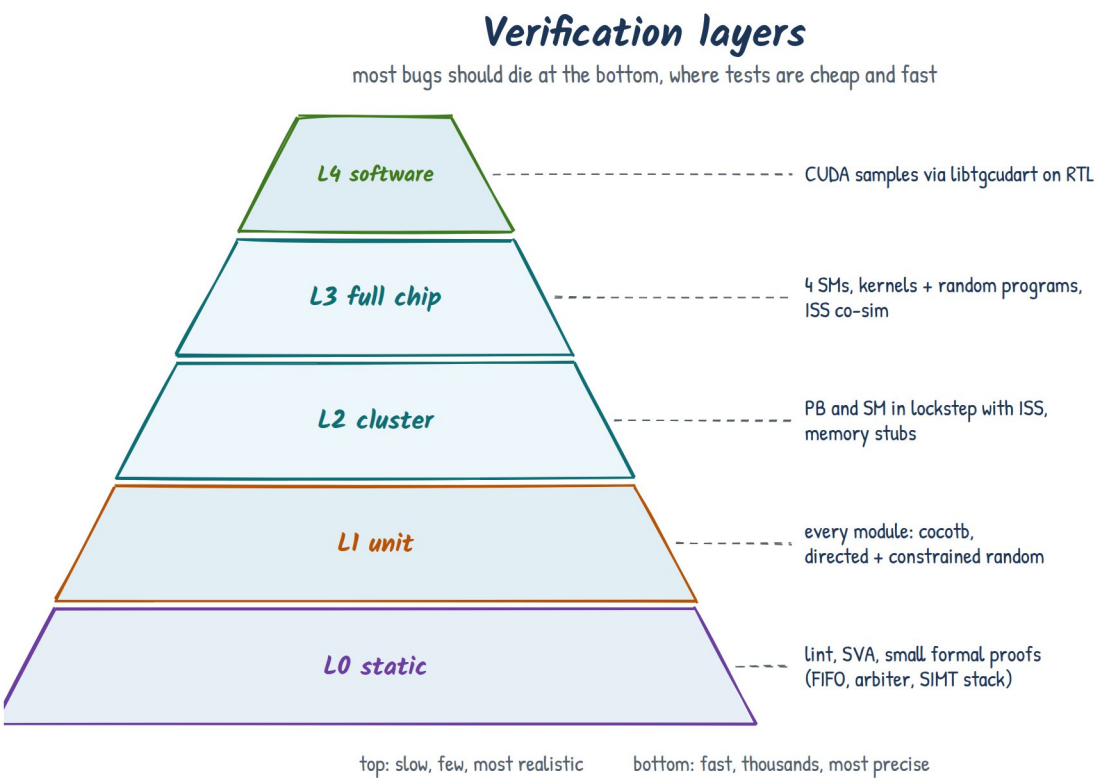


Figure 21. Verification layers from static checks to software.

Chip-level verification environment

lockstep comparison against the golden ISS, plus end-of-test memory compare

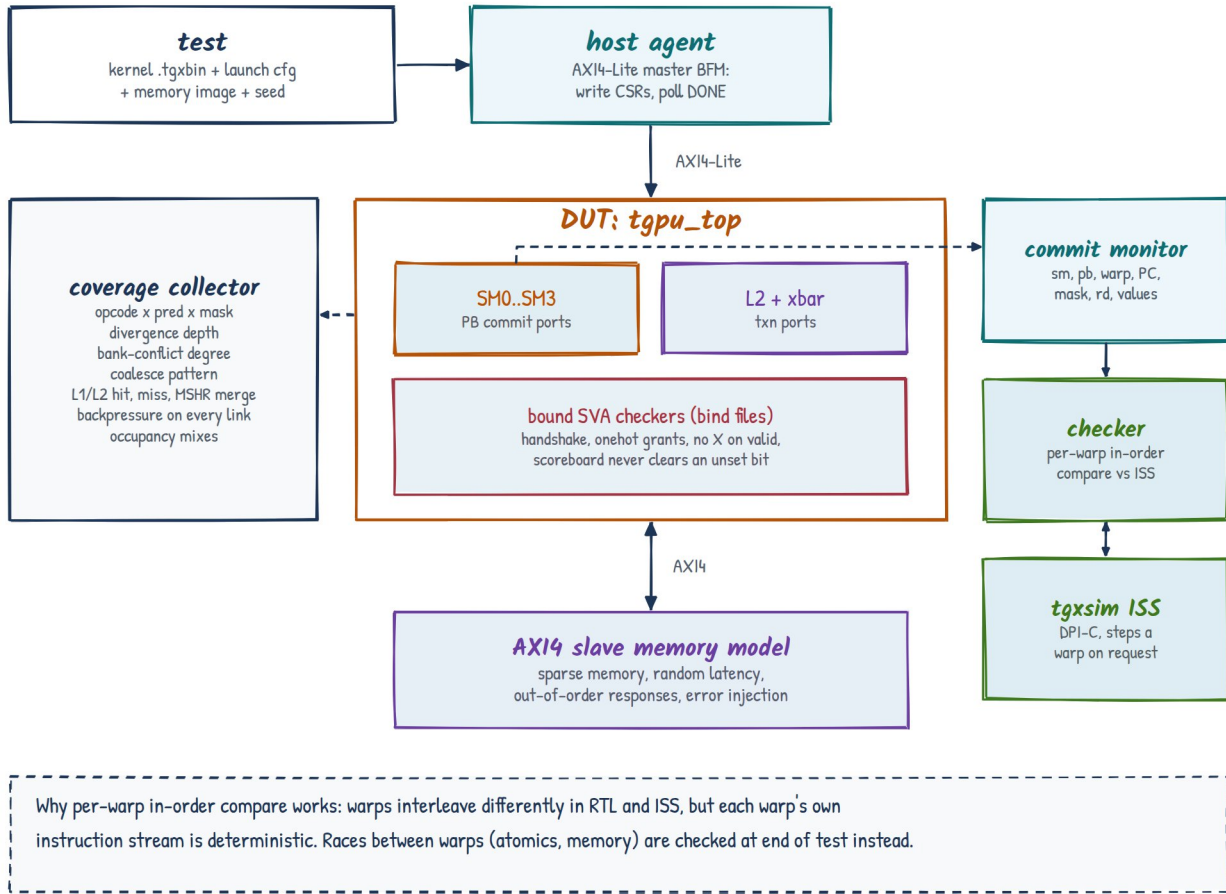


Figure 22. Chip-level environment: host agent, AXI memory model, commit monitor, ISS lockstep checker, SVA and coverage.

14.1 Strategy

- Golden model first: txgsim is a functional, warp-level ISS written from the spec. It executes the same tgxbin and produces a commit log (sm, pb, warp, PC, mask, rd, 32 lane values, predicate writes, memory writes).
- Unit level: cocotb (<https://www.cocotb.org/>) with Verilator. Python reference models for each block. Directed tests for every corner, then constrained random with random back-pressure on every valid/ready port.
- Cluster and chip level: a C++ or SV testbench with DPI-C to txgsim. Per-warp in-order commit compare (warps interleave differently, but each warp stream is deterministic). Memory and atomics compared at end of test against the ISS final memory image; for atomics compare the final value and the multiset of returned old values.
- Assertions: SVA modules attached with bind so RTL stays clean. Small formal proofs on FIFO, skid buffer, round-robin arbiter, SIMT stack and scoreboard where the tools allow.
- Random program generator: produces legal TGX programs with bounded loops, nested divergence, barriers only in convergent code, and memory accesses inside a sandboxed address range.

14.2 Unit test plan

Block	Key tests	Checker
lib_fifo / lib_skid	full, empty, simultaneous push/pop, random stall	Python queue model + SVA

lib_rr_arb	fairness, starvation bound, one-hot grant	SVA: grant within N cycles
tgpu_alu_int	all ops, all lanes, lane masks, signed/unsigned edges	Python model
tgpu_fpu	RNE ties, overflow to inf, FTZ, NaN, signed zero, 1e7 random	SoftFloat via ctypes
tgpu_scoreboard	RAW/WAW per warp, same-cycle set and clear	model + SVA never clears unset bit
tgpu_simt_stack	uniform, divergent, nested 8 deep, overflow, EXIT in branch	Python stack model
tgpu_warp_sched	all ready patterns, fairness, GTO greediness	model
tgpu_rf	all rows, lane-masked writes, read-during-write	memory model
tgpu_lsu	every coalescing pattern, 1 to 8 way conflicts, broadcast, partial masks	address model
tgpu_l1d / tgpu_l2_bank	hit, miss, evict, MSHR merge, MSHR full, dirty writeback, atomics	cache model + memory compare
tgpu_xbar	all-to-one hot spot, all-to-all, back-pressure	transaction scoreboard
tgpu_cta_dispatch / cta_mgr	resource-limited mixes, fragmentation, grid of 1 and of 10000	occupancy model

14.3 Kernel suite (chip level and software level)

Kernel	Exercises
vecAdd, SAXPY	basic launch, global loads and stores, coalescing
divergent if/else, nested loops with break	SIMT stack, reconvergence
parallel reduction (shared memory)	LDS/STS, BAR.SYNC, bank conflicts
matrix transpose (naive vs padded)	bank conflicts and their fix
tilled SGEMM 64x64	FFMA throughput, shared memory reuse, L1/L2
histogram	global and shared atomics, contention
prefix scan	barriers in loops, multi-phase
grid larger than the machine	dispatcher refill, CTA retire
random TGX programs	everything; the main bug finder

14.4 Functional coverage model

- Every opcode x guard predicate (true, false, PT, negated) x active-mask class (all, none, one, partial).
- SIMT stack depth reached 0..8, overflow, EXIT inside divergent code.
- Bank conflict degree 1..8 and broadcast; coalescing: 1, 2, 4, 8..32 lines per warp.
- L1 and L2: hit, miss, eviction clean/dirty, MSHR merge, MSHR full stall, atomics on the same line back to back.
- Back-pressure asserted on every valid/ready link at least once while valid is high.
- Occupancy limited by each resource (warps, regs, smem, CTA slots); all 4 SMs busy simultaneously.
- Scheduler: every warp granted; every stall reason observed (scoreboard, barrier, unit busy, l-buffer empty, MEMBAR).

14.5 Assertion checklist

- Handshake: valid stable until ready; data stable while valid && !ready; no X on valid or ready after reset.
- Arbiters: grant is one-hot or zero, grant implies request, bounded wait.
- Scoreboard: no issue with a pending source; no clear of an unset bit.
- SIMT: active mask subset of the warp live mask; never pop an empty stack.
- Memory: every request gets exactly one response; MSHR count never exceeds 8; no two valid lines with the same tag in one set.
- Dispatcher: allocated resources never exceed capacity; every dispatched CTA eventually reports done (liveness in simulation with a timeout).

14.6 Regression and sign-off

When	What runs	Pass criteria
every commit (CI)	lint, all unit smoke tests, vecAdd on RTL	zero failures, zero new lint warnings
nightly	full unit random seeds, kernel suite, 1000 random	no failures; coverage trend not decreasing

	programs, coverage merge	
milestone	everything + long random soak (1e8 cycles)	milestone exit criteria in Section 12
FE sign-off	RTL freeze candidate	100% of planned functional coverage, >95% line/branch coverage with reviewed exclusions, zero open bugs of severity high, synthesis clean

Debug aids to build early: a commit-log diff tool (first mismatching warp and instruction), a waveform dump window around the mismatch, per-warp PC trace, and perf counters readable over CSR (they double as coverage evidence).

15. Glossary

Term	Meaning
SM / CU	streaming multiprocessor / compute unit: schedulers + execution + local storage
PB	processing block: one scheduler, its RF and lanes (a quarter-SM in NVIDIA terms)
CTA	cooperative thread array = thread block
warp	32 threads executing in lockstep with a shared PC
lane	one ALU slot; not a software thread
beat	one pass of 8 lanes; a warp instruction takes 4 beats
IPDOM	immediate post-dominator, the reconvergence point after a branch
MSHR	miss status holding register, tracks one outstanding cache miss
coalescing	merging lane addresses into as few cache-line requests as possible
occupancy	resident warps / maximum warps per SM
ISS	instruction set simulator, the golden functional model

16. References

- NVIDIA PTX ISA: <https://docs.nvidia.com/cuda/parallel-thread-execution/>
- CUDA Programming Guide, SIMT kernels and bank conflicts: <https://docs.nvidia.com/cuda/cuda-programming-guide/02-basics/writing-cuda-kernels.html>
- Compiling CUDA with clang: <https://llvm.org/docs/CompileCudaWithLLVM.html>
- GPGPU-Sim manual (SIMT stack, operand collector): https://gpgpu-sim.org/manual/index.php/Main_Page
- Vortex open-source GPGPU: <https://github.com/vortexgpgpu/vortex>
- CVFPU / FPnew SystemVerilog FPU: <https://github.com/openhwgroup/cvfpu>
- Berkeley SoftFloat: <https://github.com/ucb-bar/berkeley-softfloat-3>
- cocotb: <https://www.cocotb.org/>
- Verilator: <https://verilator.org/guide/latest/overview.html>